

Модель given → when → then

GIVEN
подготовкаWHEN
действиеTHEN
проверка

```
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;

given()
    .baseUrl("https://restful-booker.herokuapp.com")
    .accept(ContentType.JSON)
    .when()
    .get("/booking")
    .then()
    .statusCode(200)
    .contentType(ContentType.JSON)
    .body("bookingid", everyItem(greaterThan(0)));
```

Что находится в каждом блоке

given()	base URI, headers, cookies, path/query params, body, filters, auth.
when()	HTTP-метод: get/post/put/patch/delete и endpoint.
then()	Status, content type, headers, body, schema, extraction.

Правило: подготовка не должна содержать assertion, а проверка — выполнять новый бизнес-запрос.

Основные HTTP-методы

GET	Чтение. Обычно без body.	POST	Создание/команда.
PUT	Полная замена.	PATCH	Частичное изменение.
DELETE	Удаление.	HEAD	Только headers.

Минимум полезных проверок

- **Status:** соответствует контракту, а не просто «любой 2xx».
- **Content-Type:** клиент действительно получил ожидаемый формат.
- **Структура:** обязательные поля существуют и имеют нужный тип.
- **Данные:** ответ относится именно к отправленному запросу.
- **Эффект:** после POST/PUT/PATCH состояние подтверждено отдельным GET.

Полезные проверки body

```
.body("firstname", equalTo("Ivan"))
.body("totalprice", greaterThan(0))
.body("depositpaid", is(true))
.body("bookingdates.checkin", notNullValue())
.body("bookingid", hasSize(greaterThan(0)))
```

Частые matchers

equalTo notNullValue hasItem hasSize contains everyItem
greaterThan allOf anyOf

Health check Restful Booker

```
given()
    .baseUrl(BASE_URL)
    .when()
    .get("/ping")
    .then()
    .statusCode(201)
    .body(equalTo("Created"));
```

Не предполагайте, что health endpoint обязан вернуть 200: проверяйте документированный контракт.

Антипаттерны первого уровня

- Только statusCode(200) без данных и структуры.
- Сборка URL строкой: "/booking/" + id, когда есть path param.
- Thread.sleep вместо ожидания условия.
- Случайный booking id из общего списка для PUT/DELETE.
- assertThat(true).isTrue() после непроверенных вычислений.
- Один тест зависит от результата другого теста.

Быстрый шаблон нового теста

1. Подготовить уникальные данные.
2. Выполнить ровно одно проверяемое действие.
3. Проверить transport contract.
4. Проверить бизнес-значения.
5. Очистить созданные данные даже при падении.

Path и query parameters

```
given()
  .baseUrl(BASE_URL)
  .pathParam("id", bookingId)
.when()
  .get("/booking/{id}");

given()
  .baseUrl(BASE_URL)
  .queryParams("firstname", "Ivan")
  .queryParams("lastname", "Petrov")
.when()
  .get("/booking");
```

Path определяет ресурс, **query** фильтрует/сортирует/задаёт страницу. REST Assured сам выполнит URL encoding.

RequestSpecification

```
RequestSpecification baseSpec =
  new RequestSpecBuilder()
    .setBaseUrl(BASE_URL)
    .setAccept(ContentType.JSON)
    .setContentType(ContentType.JSON)
    .addFilter(new ErrorLoggingFilter())
    .build();

Response response = given()
  .spec(baseSpec)
  .body(request)
  .when()
  .post("/booking");
```

Храните здесь общие технические настройки. Не кладите специфичный expected firstname или изменяемый booking id.

Headers, cookies и auth

```
given()
  .header("X-Request-Id", requestId)
  .accept(ContentType.JSON)
  .contentType(ContentType.JSON)
  .cookie("token", token)
.when()
  .put("/booking/{id}", id);
```

- Accept — какой ответ понимает клиент.
- Content-Type — формат отправляемого body.
- Не логируйте token/password в рабочем CI.

ResponseSpecification

```
ResponseSpecification okJson =
  new ResponseSpecBuilder()
    .expectStatusCode(200)
    .expectContentType(ContentType.JSON)
    .build();

given().spec(baseSpec)
  .when().get("/booking")
  .then()
    .spec(okJson)
    .body("bookingid", everyItem(greaterThan(0)));
```

Specification проверяет общий transport contract; конкретные бизнес-ожидания остаются в тесте.

Body: Map → DTO

```
// Быстро, но слабее типизация
Map<String, Object> body = Map.of(
  "firstname", "Ivan",
  "totalprice", 150);

// Предпочтительно для поддерживаемого suite
record BookingDates(String checkin, String checkout) {}
record BookingRequest(
  String firstname, String lastname,
  int totalprice, boolean depositpaid,
  BookingDates bookingdates,
  String additionalneeds) {}

given().contentType(JSON).body(bookingRequest);
```

Когда какой body

String	Короткий негативный JSON, проверка «сырого» payload.
Map	Маленький PATCH или одноразовое тело.
DTO	Основной путь: типы, IDE, builder, рефакторинг.
File	Большой статичный fixture; следить за актуальностью.

Form, multipart и массив query

```
given().formParam("username", "ivan");
given().multiPart("file", file, "image/png");
given().queryParams("status", List.of("NEW", "PAID"));
```

Точный wire-format массива зависит от API: повторяющиеся keys, CSV или brackets. Проверяйте документацию/traffic.

Где чаще всего ошибаются

- Путают Accept и Content-Type.
- Глобально изменяют RestAssured.baseUrl в параллельных тестах.
- Повторяют 10 строк настройки в каждом тесте.
- Создают «универсальную spec» со всеми возможными headers.
- Хранят production credentials в исходниках.
- DTO не соответствует именам JSON-полей и silently получает null.

Практический ориентир

Map для небольшого PATCH. **DTO** для create/PUT и чтения. **Specification** для одинаковой транспортной настройки. **Builder/factory** для удобного создания тестовых данных.

Четыре способа работать с ответом

```
// 1. Inline contract
given().when().get("/booking/{id}", id)
  .then().statusCode(200)
  .body("firstname", equalTo("Ivan"));

// 2. Нужен весь HTTP response
Response response = given().when().get(...);
int status = response.statusCode();

// 3. Одно значение для следующего шага
int id = response.then().extract().path("bookingid");

// 4. Типизированный объект
BookingResponse dto = response.as(BookingResponse.class);
```

Headers и время ответа

```
response.then()
  .header("Content-Type", containsString("json"))
  .header("X-Request-Id", not(blankString()))
  .time(lessThan(2000L));
```

Жёсткая проверка latency в shared test-среде часто flaky. Для performance SLA нужны специализированные нагрузочные тесты и статистика, а не один запрос.

JSON Schema

```
given().spec(baseSpec)
  .when().get("/booking/{id}", id)
  .then()
    .statusCode(200)
    .body(matchesJsonSchemaInClasspath(
      "schemas/booking-schema.json"));
```

Schema проверяет: required, type, format, nested structure.
Не проверяет: правильное бизнес-значение firstname или допустимость дат.

DTO + AssertJ: удобно для многих полей

```
BookingResponse actual = response.as(BookingResponse.class);

assertThat(actual)
  .usingRecursiveComparison()
  .isEqualTo(expected);

assertThat(actual.totalprice()).isPositive();
assertThat(actual.bookingdates().checkout())
  .isGreaterThan(actual.bookingdates().checkin());
```

Recursive comparison проверяет данные, но бизнес-инварианты добавляйте отдельно. Не игнорируйте поля только ради зелёного теста.

Что выбрать

Задача	Инструмент
2–4 простых поля	then().body(...)
ID для следующего запроса	extract().path()
Status + headers + raw body	Response
Много связанных полей	DTO + AssertJ
Форма контракта	JSON Schema
Сложная коллекция	DTO/List + AssertJ extracting

JsonPath / GPath

```
response.jsonPath().getString("firstname");
response.jsonPath().getInt("totalprice");
response.jsonPath().getList("bookingid", Integer.class);

// Фильтрация массива (Groovy GPath)
.body("items.findAll { it.price > 100 }.name",
  hasItems("Book", "Keyboard"));
```

Если выражение трудно прочитать вслух — извлеките коллекцию и проверяйте AssertJ.

Слабые проверки

Плохо

body("id",
notNullValue())
для созданного объекта.

Лучше

Извлечь id, GET по нему и сверить отправленные поля.

Плохо

Сравнить весь JSON как строку.

Лучше

Schema + значимые поля или DTO.

Типы: частая ловушка

- JSON number может стать Integer, Long, Float или BigDecimal.
- Не сравнивайте денежные double точным равенством.
- Для дат десериализуйте LocalDate или явно проверяйте ISO-формат.
- Отличайте отсутствующее поле от поля со значением null.

Проверка ошибки

```
response.then()
  .statusCode(400)
  .contentType(ContentType.JSON)
  .body("code", equalTo("VALIDATION_ERROR"))
  .body("fieldErrors.fieldId", hasItem("checkin"));
```

Негативный тест проверяет не только status, но и стабильный machine-readable error contract.

1. Получить token

```
String token = given()
    .spec(baseSpec)
    .contentType(ContentType.JSON)
    .body(Map.of(
        "username", "admin",
        "password", "password123"))
    .when()
    .post("/auth")
    .then()
    .statusCode(200)
    .body("token", not(blankString()))
    .extract().path("token");
```

2. Создать и извлечь id

```
int id = given()
    .spec(baseSpec)
    .contentType(ContentType.JSON)
    .body(request)
    .when()
    .post("/booking")
    .then()
    .statusCode(200)
    .body("booking.firstname",
        equalTo(request.firstname()))
    .extract().path("bookingid");
```

3. GET → PUT → PATCH

```
// Подтвердить создание
BookingResponse saved = given().spec(baseSpec)
    .get("/booking/{id}", id).as(BookingResponse.class);
assertThat(saved.firstname()).isEqualTo(request.firstname());

// Полная замена
given().spec(baseSpec).cookie("token", token)
    .contentType(JSON).body(fullUpdate)
    .when().put("/booking/{id}", id)
    .then().statusCode(200);

// Частичное изменение
given().spec(baseSpec).cookie("token", token)
    .contentType(JSON).body(Map.of("lastname", "New"))
    .when().patch("/booking/{id}", id)
    .then().statusCode(200);
```

После mutation выполните GET: response на PUT/PATCH может быть правильным, а сохранение — сломанным.

PUT vs PATCH

PUT	Передайте полное представление. Проверьте все значимые поля.
PATCH	Передайте изменяемые поля. Проверьте изменение и сохранность остальных.

4. Cleanup в finally

```
Integer id = null;
try {
    id = createBooking(request);
    verifyBooking(id, request);
    patchBooking(id, token);
    verifyPatch(id);
} finally {
    if (id != null) {
        given().spec(baseSpec)
            .cookie("token", token)
            .when()
            .delete("/booking/{id}", id);
    }
}
```

Cleanup запускается даже после упавшей проверки. Его ошибка не должна затереть исходную причину падения.

5. Подтвердить удаление

```
given().spec(baseSpec)
    .cookie("token", token)
    .when()
    .delete("/booking/{id}", id)
    .then()
    .statusCode(201);

given().spec(baseSpec)
    .when()
    .get("/booking/{id}", id)
    .then()
    .statusCode(404);
```

Правила независимости

- Каждый mutation-тест создаёт собственную запись.
- Не берите первый id из общего GET /booking.
- Уникальное имя помогает найти запрос в логах.
- Не делайте create/update/delete отдельными зависимыми @Test.
- Одна CRUD-цепочка допустима внутри одного end-to-end теста.
- Если API eventually consistent — ждите условие с timeout, не sleep.

Негативная auth-матрица

Сценарий	Что проверить
Нет cookie	Status + error contract.
Неверный token	Отказ, состояние не изменилось.
Пустые credentials	Предсказуемая validation/auth error.
Обычный user	Недостаточно прав на admin action.

Shared test API

Restful Booker общий и периодически сбрасывается. Не утверждайте точный размер списка, не используйте заранее известный id и не считайте исчезновение чужой записи дефектом вашего теста.

Разделение ответственности

Слой	Отвечает за
Test	Сценарий и ожидаемое бизнес-поведение.
API client	Endpoint, method, transport; возвращает Response/DTO.
DTO	Контракт request/response.
Data factory	Валидные уникальные данные и варианты.
Specifications	Общие HTTP-настройки и общий transport contract.
Assertions	Переиспользуемый доменный язык проверок.

Client выполняет запрос, test определяет ожидание. Не прячьте все assertions внутрь клиента.

Checklist перед merge

- Название объясняет условие и ожидаемый результат.
- Тест независим и создаёт свои данные.
- Status code взят из контракта.
- Проверены content type и ключевые поля.
- Mutation подтверждён отдельным чтением состояния.
- Негативный тест проверяет machine-readable error.
- Path/query params не собираются вручную строкой.
- Общие настройки вынесены в specification.
- Секреты и PII не попадают в logs/Allure.
- Cleanup выполняется при любом исходе.
- Нет sleep, зависимости от порядка и случайного чужого id.
- Падение сообщает expected, actual и контекст запроса.

Условное логирование

```
given()
  .spec(baseSpec)
  .log().ifValidationFails()
.when()
  .get("/booking/{id}", id)
.then()
  .log().ifValidationFails()
  .statusCode(200);
```

- Прикладывайте request/response в Allure при падении.
- Маскируйте Authorization, Cookie, password, PII.
- Логируйте correlation/request id.
- Глобальный `log().all()` создаёт шум и утечки.

Пирамида проверок одного response

1. **Transport:** status, content type, нужные headers.
2. **Contract:** required fields, types, schema.
3. **Data:** ответ соответствует request/test fixture.
4. **Business:** цены, даты, статусы, права, инварианты.
5. **State:** повторное чтение подтверждает mutation.

Не каждая проверка нужна каждому endpoint, но пропуск уровня должен быть осознанным.

Быстрый шаблон диагностики падения

```
Что ожидали: PATCH меняет только lastname
Что получили: firstname также изменился
Request: method, path, safe headers, body
Response: status, headers, body, time
State GET: фактически сохранённый DTO
Environment: URL, build, correlation id
Вывод: product / test / environment defect
```

Timeout, retry, eventual consistency

- Задавайте connect/read timeout на уровне HTTP config.
- Retry допустим для явно транзIENTНОЙ инфраструктурной ошибки.
- Не retry-те assertion до зелёного: это маскирует дефект.
- Для async-состояния используйте polling с общим timeout и диагностикой последнего ответа.
- POST может быть неидемпотентным: слепой retry создаст дубликаты.

Признаки хрупкого теста

- Порядок тестов имеет значение.
- Общий изменяемый token/id/body.
- Точные latency assertions на shared environment.
- Длинный GPath дублирует production-алгоритм.
- Десятки игнорируемых полей recursive comparison.
- Cleanup отсутствует или может скрыть исходное падение.

Достаточный уровень middle+

Вы не только пишете зелёный запрос, но можете:

- объяснить границы теста и выбранные assertions;
- спроектировать переиспользуемый client/spec/DTO;
- локализовать transport, contract и product defect;
- сделать тест параллельным, независимым и диагностируемым;
- отличить полезный retry от маскировки нестабильности;
- провести code review API-теста по checklist выше.